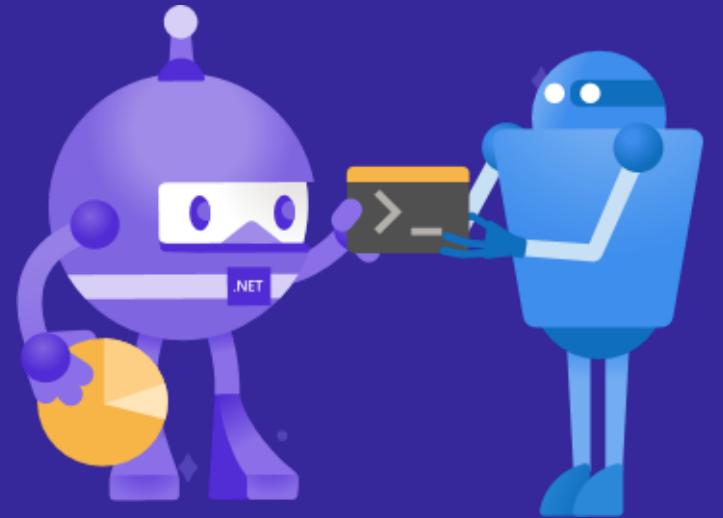


대 AI 시대에도 DB 는 필요하다.

ORM 딱 정해준다!

- *EF vs Dapper* ORM 대전! -

이성환





이성환 (Greg) greg@clo3d.com
(주)클로버추얼패션

.NET 외길 인생

C#, XAML,

Powershell

WPF, MVVM

Winform,

WCF,

ASP.NET

MS-SQL etc...

CONTENTS

- 서론(밑밥깔기)
- **ORM** 이란?
- 본격 단점 꼬집기!
- 그래서 당신의 선택은?
- 이제 해야 할 일들...

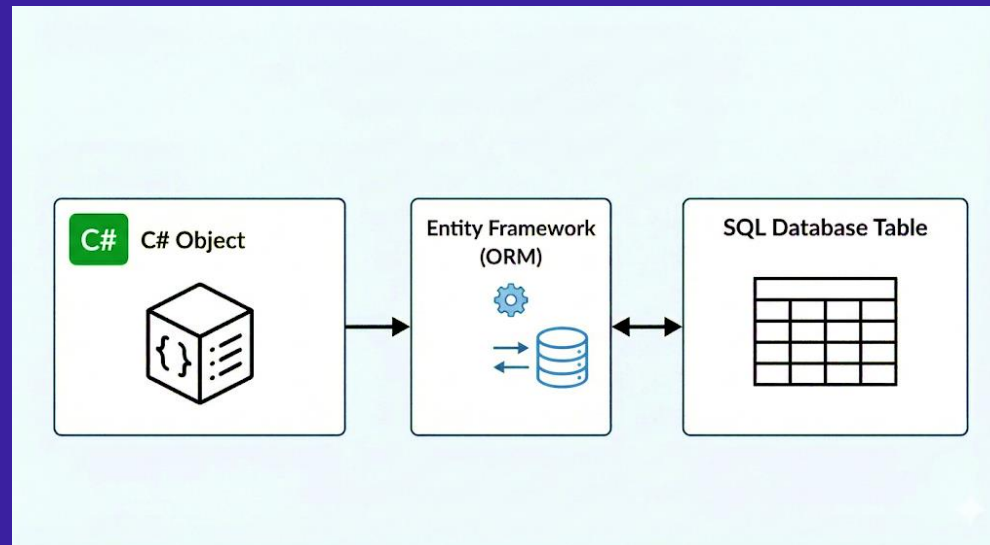


ORM 이란?

ORM 이란?

- **ORM** (Object-Relational Mapping)

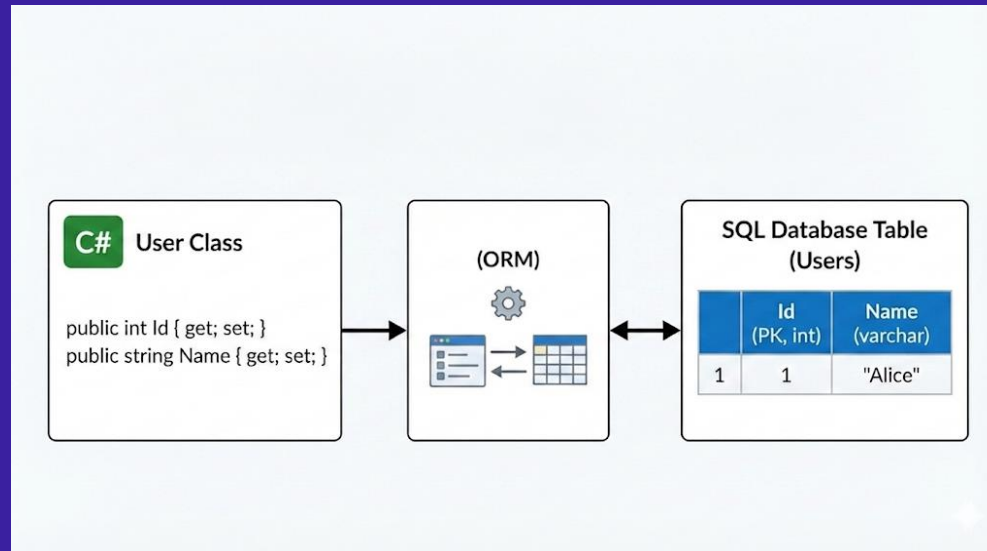
프로그래밍 언어의 객체 ↔ 데이터베이스의 테이블 매핑



- **ORM** (Object-Relational Mapping)

핵심은 바로 DB 의 테이블, 레코드 따위를 C#의 타입과 맵핑하는 것!

타입!



- .NET의 주요 ORM 도구

- Full ORM

Entity Framework (Core)

NHibernate

- Micro ORM

Dapper

RepoDb

Entity Framework (EF)

- **Entity Framework (core)**

- Microsoft의 공식 ORM 프레임워크 (Full ORM, 토탈패키지!)
- .NET의 표준 데이터 액세스 기술
- EF6의 완전한 재작성 (크로스 플랫폼)
- 현재 최신 버전: EF Core 10 (.NET 10)
- 닷넷 표준이지만 벤더 독립적 구성(고수준 추상화 제공)

• 핵심 개념과 철학 (+ 장점)

> 완전한 추상화

Full ORM. (DB 를 통째로 app 에 맵핑)

LINQ로 모든 쿼리 표현

IQueryable 을 활용한 추상화 제공

데이터베이스 벤더 독립적

Provider	패키지
SQL Server	Microsoft.EntityFrameworkCore.SqlServer
PostgreSQL	Npgsql.EntityFrameworkCore.PostgreSQL
MySQL	Pomelo.EntityFrameworkCore.MySql
SQLite	Microsoft.EntityFrameworkCore.Sqlite
Oracle	Oracle.EntityFrameworkCore
Cosmos DB	Microsoft.EntityFrameworkCore.Cosmos

- 핵심 개념과 철학 (+ 장점)

- > "Code First" 접근법

- C# 클래스로 데이터 모델 정의

- 데이터베이스는 자동 생성/관리

- 객체 지향적 사고 유지

- 간단 샘플

```
var activeUsers = await _context.Users
    .Where(u => u.IsActive && u.CreatedAt > DateTime.Now.AddMonths(-6))
    .OrderByDescending(u => u.LastLoginAt)
    .Take(10)
    .ToListAsync();

var user = await _context.Users.FindAsync(userId);
user.Email = newEmail;
user.UpdatedAt = DateTime.UtcNow;

await _context.SaveChangesAsync();
```

Dapper

• Dapper

- StackOverflow가 만든 Micro ORM(since 2011)
- ADO.NET 확장 라이브러리
- SQL 은 string, DB 응답은 .net 의 타입으로 맵핑
- framework 수준보다는 library 로 기능을 제공.
- 현재 GitHub Star 18k+

• 지향점

개발자 : SQL 작성

ADO.NET : SQL 실행

Dapper : 객체 매핑

- 간단샘플

```
var orders = await connection.QueryAsync<Order, Customer, OrderItem, Order>(
    @"SELECT o.*, c.*, oi.*
      FROM Orders o
     INNER JOIN Customers c ON o.CustomerId = c.Id
     INNER JOIN OrderItems oi ON o.Id = oi.OrderId
    WHERE o.OrderDate >= @startDate",
    (order, customer, orderItem) => {
        order.Customer = customer;
        order.OrderItems.Add(orderItem);
        return order;
    },
    new { startDate },
    splitOn: "Id,Id"
);
```

본격 단점 꼬집기

- EF -

• EF 단점?

All in one 패키지! (DB first)

All or Nothing

레거시 통합의 번거로움. (당연히 DB first)

IQueryable<T>는 추상화가 아니야?

비즈니스 로직의 분리? (의식의 흐름대로..)

DB 호출 통합 테스트는 어떻게??

SQL 확인은? 그래서 그게 C# 코드 어딘데?

.....

- All in one 패키지! (Database first)
 - DB 동기화가 필요.
 - (수 백, 수 천 개) 테이블을 전부 C# 클래스로 변환
 - DbContext 가 거대해지고 초기화 시간 증가
 - DB context 생성 방식도 가지가지, 종종 에러가 발생하기도.
(scaffolding, PowerTools, T4..)

• All or Nothing

- 구조적 관점에서

부분 사용 여부와 관계없이 전체 구조를 준비하고 실행해야함. (all in one)

예1> 컬럼 50개 테이블에서 1개 값만 가져오고 싶지만(select) 테이블 전체 컬럼 정의가 필요.

예2> 전체 테이블 1000개 중 내가 쓰는 테이블이 1개뿐이지만 전체 테이블 정의가 필요.

- 트랜잭션 관점에서

DbContext 를 단일 트랜잭션으로 고정. (Unit of work)

SaveChanges() 로 원자적 단위(Atomic boundary) 설정.

모두 성공하면 커밋!

하나라도 실패하면 전부 롤백!

- All or Nothing (트랜잭션 관점에서)

정상 상황에서 문제가 되지 않음.

예외 상황에서 문제가 되거나 비효율적.

```
var newItem = items.ToList(); // 대략 1000 건

if (newItem.Any())
{
    await _context.AddRangeAsync(newItem);
    await _context.SaveChangesAsync(); // insert 과정에서 1건이라도 실패하면?
}
```

• 레거시 통합의 번거로움 (Database first)

서비스 플랫폼 변경이나 전환 시도 시

```
// 20년 된 DB의 실제 테이블명과 컬럼명
// tbl_usr_mst, usr_id, usr_nm, reg_dt, upd_dt...

// EF에서 매핑하려면
[Table("tbl_usr_mst")]
public class User
{
    [Column("usr_id")]
    public int Id { get; set; }

    [Column("usr_nm")]
    public string Name { get; set; }

    // 50개 컬럼 x 100개 테이블 = 지옥
}
```

• IQueryable<T>는 추상화가 아니야?

누수된 추상화(Leaky Abstraction)

- 추상화가 내부 구현을 완전히 감추지 못하고,
외부 코드가 내부 동작을 알아야 정상적으로 쓸 수 있는 상태
- 추상화의 가치는 객체로 주고받을 수 있을 때 더 강화됨

근데 그게 왜?

• IQueryable<T>

```
public IQueryable<User> GetActiveUsers()  
{  
  
    return _context.Users  
        .Where(u => u.IsActive)  
        .Where(u => u.HasOrdered);  
}
```

```
// Service Layer  
public async Task<List<UserDto>> GetUsersWithOrders()  
{  
    var users = _repository.GetActiveUsers()  
        .Include(u => u.Orders)           // 추가 조인  
        .ThenInclude(o => o.OrderItems)  // 더 추가  
        .Where(u => u.Orders.Any())       // 추가 필터  
        .OrderByDescending(u => u.CreatedAt) // 정렬 변경  
        .Take(100);                      // 제한 추가  
  
    return await users.ToListAsync();  
}
```

• IQueryable<T>

```
IQueryable<User> query = dbContext.Users;

if (request.IsActive is true)
{
    query = query.Where(x => x.IsActive);
}

if (request.MinAge is not null)
{
    query = query.Where(x => x.Age >= request.MinAge);
}

if (string.IsNullOrEmpty(request.Name) is false)
{
    query = query.Where(x => x.Name.Contains(request.Name));
}

List<User> result = await query.ToListAsync();
```

```
public static class UserQueryExtensions
{
    public static IQueryable<User> Active(this IQueryable<User> query)
    {
        return query.Where(x => x.IsActive);
    }

    public static IQueryable<User> OlderThan(this IQueryable<User> query, int age)
    {
        return query.Where(x => x.Age >= age);
    }
}
```

• 비즈니스 로직의 분리? (의식의 흐름대로..)

주로 DbContext 를 DI 로 주입해 사용.

- Repository 패턴을 사용하지 않는 경우가 많음.
- DB 조작을 의식의 흐름대로...

그래서 그게 왜?

◆비즈니스 로직의 분리? (의식의 흐름대로..)

레이어 붕괴

: DbContext의 무분별한 침투

```
// Controller가 DB 로직 직접 처리
[ApiController]
[Route("api/orders")]
public class OrderController(AppDbContext context) : ControllerBase
{
    [HttpPost]
    public async Task<IActionResult> CreateOrder(CreateOrderRequest request)
    {
        var customer = await context.Customers
            .FirstOrDefaultAsync(c => c.Id == request.CustomerId);

        // 재고 확인
        var product = await context.Products
            .FirstOrDefaultAsync(p => p.Id == request.ProductId);

        // 할인 계산
        var discount = customer.IsVip ? 0.1m : 0m;
        var totalAmount = product.Price * request.Quantity * (1 - discount);

        // 주문 생성
        _context.Orders.Add(new Order { CustomerId = customer.Id, TotalAmount = totalAmount });

        // 재고 감소
        product.Stock -= request.Quantity;

        await _context.SaveChangesAsync();

        return Ok(order);
    }
}
```

◆비즈니스 로직의 분리? (의식의 흐름대로..)

비즈니스 (or Domain) 레이어 오염

: 비즈니스 로직에서 DbContext 직접 사용

```
public class OrderService(AppDbContext context, IEmailService emailService)
{
    public async Task<Result> ProcessOrder(int orderId)
    {
        // 비즈니스 로직과 DB 쿼리가 뒤섞임
        var order = await _context.Orders
            .Include(o => o.Customer)
            .Include(o => o.OrderItems)
            .ThenInclude(oi => oi.Product)
            .FirstOrDefaultAsync(o => o.Id == orderId);

        // 비즈니스 규칙
        if (order.Status != OrderStatus.Pending) return Result.Fail("이미 처리된 주문입니다.");

        // 결제 처리
        var payment = new Payment { OrderId = orderId, Amount = order.TotalAmount };
        _context.Payments.Add(payment);

        await _context.SaveChangesAsync();

        // 이메일 발송
        await _emailService.SendOrderConfirmation(order);

        return Result.Success();
    }
}
```

- DB 호출 통합 테스트는 어떻게?

```
var report = await _context.Orders
    .Where(o => o.OrderDate >= new DateTime(2024, 1, 1))
    .GroupBy(o => new { o.OrderDate.Year, o.OrderDate.Month })
    .Select(g => new { g.Key.OrderNo, g.Key.UserId, g.Key.OrderDate })
    .OrderByDescending(r => r.Year)
    .ThenByDescending(r => r.Month)
    .ToListAsync();
```

솔직히 우리는 EF 가 어떤 SQL 을 생성할 지 알 수 없다.

근데 알고 싶으면 어떡함?

```
var monstrousQuery = await _context.Orders
    .Where(o => o.OrderDate >= new DateTime(2024, 1, 1))
    .Join(_context.Customers, o => o.CustomerId, c => c.Id, (o, c) => new { o, c })
    .Join(_context.CustomerGrades, oc => oc.c.GradeId, cg => cg.Id, (oc, cg) => new { oc.o, oc.c, cg })
    .Join(_context.Shippers, occg => occg.o.ShipperId, s => s.Id, (occg, s) => new { occg.o, occg.c, occg.cg, s })
    .Join(_context.Employees, occgs => occgs.o.EmployeeId, e => e.Id, (occgs, e) => new { occgs.o, occgs.c, occgs.cg, occgs.s, e })
    .Join(_context.Departments, occgse => occgse.e.DepartmentId, d => d.Id, (occgse, d) => new { occgse.o, occgse.cg, occgse.s, d })
    .Select(m => new
    {
        OrderInfo = m.o,
        CustomerName = m.c.Name,
        Grade = m.cg.GradeName,
        ItemDetails = m.o.OrderDetails.Select(od => new
        {
            od.Product.ProductName,
            CategoryHierarchy = od.Product.Category.ParentCategory.Name + " > " + od.Product.Category.Name,
            StockInfo = _context.Warehouses
                .Where(w => w.Id == od.Product.WarehouseId)
                .Select(w => new {
                    w.Name,
                    LocationDetails = w.Locations
                        .Where(l => l.IsActive)
                        .Select(l => l.SectionCode)
                        .ToList()
                }).FirstOrDefault(),
            MonthlyProductSales = _context.OrderDetails
                .Where(prevOd => prevOd.ProductId == od.ProductId && prevOd.Order.OrderDate.Month == DateTime.Now.Month)
                .Sum(prevOd => prevOd.Quantity)
        })
    }).ToList()
    .OrderByDescending(x => x.OrderInfo.OrderDate)
    .ToListAsync();
```

- SQL 확인은? 그래서 그게 C# 코드 어딘데?

EF 가 어떤 SQL 을 생성하는 지 실시간으로 알기 어려움.

그리고 개발자는 **C#** 으로 모든 사고(思考)를 이어감.

이것은 **"DBA"** 와 협업하는 상황이면 더더욱 곤란.

```

-- EF Core 8.0 Generated
SELECT
    [o].[Id] AS [OrderId],
    [o].[OrderNumber],
    [c].[Name] AS [CustomerName],
    [cg].[Label] AS [Grade],
    [e].[FirstName] + N' ' + [e].[LastName] AS [Manager],
    [d].[Name] AS [Dept],
    [div].[Name] AS [Division],
    [cont].[Name] AS [Continent],
    [s].[CompanyName] AS [ShipperName],

    -- [Part 1: 12 Months Sales Trend - Subqueries 1-12]
    (SELECT SUM([o1].[TotalAmount]) FROM [Orders] AS [o1] WHERE [o1].[CustomerId] = [c].[Id] AND [o1].[OrderDate] >= '2024-01-01' AND [o1].[OrderDate] < '2024-02-01') AS [Jan_Sales],
    (SELECT SUM([o2].[TotalAmount]) FROM [Orders] AS [o2] WHERE [o2].[CustomerId] = [c].[Id] AND [o2].[OrderDate] >= '2024-02-01' AND [o2].[OrderDate] < '2024-03-01') AS [Feb_Sales],
    (SELECT SUM([o3].[TotalAmount]) FROM [Orders] AS [o3] WHERE [o3].[CustomerId] = [c].[Id] AND [o3].[OrderDate] >= '2024-03-01' AND [o3].[OrderDate] < '2024-04-01') AS [Mar_Sales],
    (SELECT SUM([o4].[TotalAmount]) FROM [Orders] AS [o4] WHERE [o4].[CustomerId] = [c].[Id] AND [o4].[OrderDate] >= '2024-04-01' AND [o4].[OrderDate] < '2024-05-01') AS [Apr_Sales],
    (SELECT SUM([o5].[TotalAmount]) FROM [Orders] AS [o5] WHERE [o5].[CustomerId] = [c].[Id] AS [o5].[OrderDate] >= '2024-05-01' AND [o5].[OrderDate] < '2024-06-01') AS [May_Sales],
    (SELECT SUM([o6].[TotalAmount]) FROM [Orders] AS [o6] WHERE [o6].[CustomerId] = [c].[Id] AND [o6].[OrderDate] >= '2024-06-01' AND [o6].[OrderDate] < '2024-07-01') AS [Jun_Sales],
    (SELECT SUM([o7].[TotalAmount]) FROM [Orders] AS [o7] WHERE [o7].[CustomerId] = [c].[Id] AND [o7].[OrderDate] >= '2024-07-01' AND [o7].[OrderDate] < '2024-08-01') AS [Jul_Sales],
    (SELECT SUM([o8].[TotalAmount]) FROM [Orders] AS [o8] WHERE [o8].[CustomerId] = [c].[Id] AND [o8].[OrderDate] >= '2024-08-01' AND [o8].[OrderDate] < '2024-09-01') AS [Aug_Sales],
    (SELECT SUM([o9].[TotalAmount]) FROM [Orders] AS [o9] WHERE [o9].[CustomerId] = [c].[Id] AND [o9].[OrderDate] >= '2024-09-01' AND [o9].[OrderDate] < '2024-10-01') AS [Sep_Sales],
    (SELECT SUM([o10].[TotalAmount]) FROM [Orders] AS [o10] WHERE [o10].[CustomerId] = [c].[Id] AND [o10].[OrderDate] >= '2024-10-01' AND [o10].[OrderDate] < '2024-11-01') AS [Oct_Sales],
    (SELECT SUM([o11].[TotalAmount]) FROM [Orders] AS [o11] WHERE [o11].[CustomerId] = [c].[Id] AND [o11].[OrderDate] >= '2024-11-01' AND [o11].[OrderDate] < '2024-12-01') AS [Nov_Sales],
    (SELECT SUM([o12].[TotalAmount]) FROM [Orders] AS [o12] WHERE [o12].[CustomerId] = [c].[Id] AND [o12].[OrderDate] >= '2024-12-01' AND [o12].[OrderDate] < '2025-01-01') AS [Dec_Sales],

```

```
-- EF Core 8.0 Generated
```

```
SELECT
```

```
    [o].[Id] AS [OrderId],
    [o].[OrderNumber],
    [c].[Name] AS [CustomerName],
    [cg].[Label] AS [Grade],
    [e].[FirstName] + N' ' + [e].[LastName] AS [Manager],
    [d].[Name] AS [Dept],
    [div].[Name] AS [Division],
    [cont].[Name] AS [Continent],
    [s].[CompanyName] AS [ShipperName],
```

이 SQL 은 도대체 어디서 만들어진 거임?

```
-- [Part 1: 12 Months Sales Trend - Subqueries 1-12]
```

```
    (SELECT SUM([o1].[TotalAmount]) FROM [Orders] AS [o1] WHERE [o1].[CustomerId] = [c].[Id] AND [o1].
[OrderDate] >= '2024-01-01' AND [o1].[OrderDate] < '2024-02-01') AS [Jan_Sales],
    (SELECT SUM([o2].[TotalAmount]) FROM [Orders] AS [o2] WHERE [o2].[CustomerId] = [c].[Id] AND [o2].
[OrderDate] >= '2024-02-01' AND [o2].[OrderDate] < '2024-03-01') AS [Feb_Sales],
    (SELECT SUM([o3].[TotalAmount]) FROM [Orders] AS [o3] WHERE [o3].[CustomerId] = [c].[Id] AND [o3].
[OrderDate] >= '2024-03-01' AND [o3].[OrderDate] < '2024-04-01') AS [Mar_Sales],
    (SELECT SUM([o4].[TotalAmount]) FROM [Orders] AS [o4] WHERE [o4].[CustomerId] = [c].[Id] AND [o4].
[OrderDate] >= '2024-04-01' AND [o4].[OrderDate] < '2024-05-01') AS [Apr_Sales],
    (SELECT SUM([o5].[TotalAmount]) FROM [Orders] AS [o5] WHERE [o5].[CustomerId] = [c].[Id] AS [o5].
[OrderDate] >= '2024-05-01' AND [o5].[OrderDate] < '2024-06-01') AS [May_Sales],
    (SELECT SUM([o6].[TotalAmount]) FROM [Orders] AS [o6] WHERE [o6].[CustomerId] = [c].[Id] AND [o6].
[OrderDate] >= '2024-06-01' AND [o6].[OrderDate] < '2024-07-01') AS [Jun_Sales],
    (SELECT SUM([o7].[TotalAmount]) FROM [Orders] AS [o7] WHERE [o7].[CustomerId] = [c].[Id] AND [o7].
[OrderDate] >= '2024-07-01' AND [o7].[OrderDate] < '2024-08-01') AS [Jul_Sales],
    (SELECT SUM([o8].[TotalAmount]) FROM [Orders] AS [o8] WHERE [o8].[CustomerId] = [c].[Id] AND [o8].
[OrderDate] >= '2024-08-01' AND [o8].[OrderDate] < '2024-09-01') AS [Aug_Sales],
    (SELECT SUM([o9].[TotalAmount]) FROM [Orders] AS [o9] WHERE [o9].[CustomerId] = [c].[Id] AND [o9].
[OrderDate] >= '2024-09-01' AND [o9].[OrderDate] < '2024-10-01') AS [Sep_Sales],
    (SELECT SUM([o10].[TotalAmount]) FROM [Orders] AS [o10] WHERE [o10].[CustomerId] = [c].[Id] AND
[o10].[OrderDate] >= '2024-10-01' AND [o10].[OrderDate] < '2024-11-01') AS [Oct_Sales],
    (SELECT SUM([o11].[TotalAmount]) FROM [Orders] AS [o11] WHERE [o11].[CustomerId] = [c].[Id] AND
[o11].[OrderDate] >= '2024-11-01' AND [o11].[OrderDate] < '2024-12-01') AS [Nov_Sales],
    (SELECT SUM([o12].[TotalAmount]) FROM [Orders] AS [o12] WHERE [o12].[CustomerId] = [c].[Id] AND
```

Dapper 단점 꼬집기

• Dapper 단점?

Code first 불가능

SQL 수작업 (by string)

1:N, N:M 은?

추상화 제공 안 됨 (DbContext, IQueryable, etc)

...

- **Code first 불가능**

너무 당연한 얘기.

Code first 는 애초에 Dapper 의 용도가 아님.

Code first 가 필요하다면 그냥 EF 를 써야함.

• SQL 수작업

- 근본적으로 SQL 을 직접 작성해야 함.
- 따로 SQL 에 대해 공부와 훈련이 필요.
- 작성한 SQL 이 옳은 지 확인 후 붙여 넣는 방식으로 사용
 - 결국 SQL 툴 필요. (+ DB 연결도 필요.)
 - 보안 이슈 발생 가능성 증가

- SQL 수작업

- SQL 을 string 으로 처리.
 - 문법 검사, 자동 완성 사용 불가
 - SQL 객체 참조 불가(검색, 이동, 수정 등)
 - 최종 mapping 타입 일치 여부 확인 불가.
 - SQL 부분 일치 부분 재사용 불가.
 - 실제 DB 와 동기화되지 않음.

Dapper 꼬집기

```
private const string OrderQuery =
    """
    SELECT
        o.Id                AS OrderId,
        c.Id                AS CustomerId,
        c.FullName          AS CustomerName,
        a.Id                AS AddressId,
        a.City,
        a.ZipCode,
        oi.Id               AS OrderItemId,
        oi.UnitPrice,
        oi.DiscountRate,
        p.Id                AS ProductId,
        p.ProductName,
        p.StockQuantity
    FROM Orders o
    INNER JOIN Customers   c  ON o.CustomerId = c.Id
    INNER JOIN Addresses   a  ON c.Id         = a.CustomerId
    INNER JOIN OrderItems  oi ON o.Id         = oi.OrderId
    INNER JOIN Products    p  ON oi.ProductId = p.Id
    WHERE o.Status = @status AND o.OrderDate >= @startDate AND o.OrderDate < @endDate
    ORDER BY
        o.OrderDate DESC,
        c.FullName  ASC
    OFFSET @offset ROWS
    FETCH NEXT @pageSize ROWS ONLY
    """;
```

• 1:N, N:M 은?

UserID	UserName	PostID	PostTitle
1	홍길동	101	첫 번째 글
1	홍길동	102	두 번째 글

```
public class User
{
    public int Id { get; set; }
    public string Name { get; set; }
    // 1:N 관계를 위한 리스트
    public List<Post> Posts { get; set; } = new List<Post>();
}

public class Post
{
    public int Id { get; set; }
    public string Title { get; set; }
    public int UserId { get; set; }
}
```

```
var result = connection.Query<User, Post, User>(
    sql,
    (user, post) =>
    {
        if (!userDictionary.TryGetValue(user.Id, out var currentUser))
        {
            currentUser = user;
            currentUser.Posts = new List<Post>();
            userDictionary.Add(currentUser.Id, currentUser);
        }

        if (post is not null)
        {
            currentUser.Posts.Add(post);
        }

        return currentUser;
    },
    param: new { UserId = 1 },
    splitOn: "Id" // 두 번째 객체(Post)의 시작 컬럼명
)
.Distinct() // 쿼리 결과 row 수만큼 반환되므로 중복 제거
.ToList();
// 최종 결과: 1개의 User 객체 안에 N개의 Posts가 들어있음
var finalUser = result.FirstOrDefault();
```

```
// EF  
var user = context.Users  
    .Include(u => u.Posts) // 이 한 줄이 위 로직 전체를 대체함  
    .FirstOrDefault(u => u.Id == 1);
```

- 추상화 제공 안 됨
 - IQueryable<T> 와 같은 추상화 제공 안 됨.
 - DB 호출을 추상화하는 기능이 없음.
 - 확정된 SQL 을 DB 로 전송하는 구조만 허용.
- transaction 을 따로 구현해야 함.(Unit of Work)

그래서 당신의 선택은?

- 결국 선택의 기준은 장점이 아니라 단점!(혹은 문제점!)

개발 시작 단계라면 오히려 더 선택하기 쉽겠지만..

나중에 문제를 만난다면??

유지할 것인가? 바꿀 것인가?

그래서 당신의 선택은?

EF 선택을 위한 질문.

(10점 이상이면 선택!)

- 개발 규모가 작다. (1)
- 간단한 쿼리만 사용한다. (1)
- 비슷한 구문이 많아서 SQL 재사용이 필수다. (2)
- 매 영속성 영역에 테스트 코드를 작성한다. (2)
- 반드시 **code first** 해야한다. (3)
- 단일 구현으로 여러 DB 를 지원해야한다. (3)
- **DB 를 직접 열어볼 일이 없다.** (5)

- 나는 EF 환자다. (10)
- 나는 C# 성애자다. (10)
- SQL 을 전혀 작성할 줄 모른다. (10)
- **DBA** 와 협업할 일이 없다. (10)

그래서 당신의 선택은?

Dapper 선택을 위한 질문.

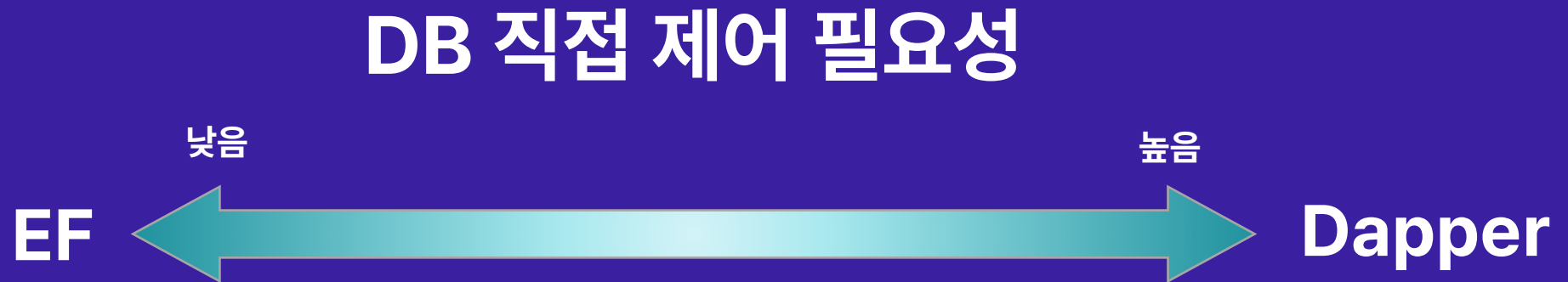
(10점 이상이면 선택!)

- 개발 조직 규모가 크다. (1)
- 단일 DB 만 사용한다. (2)
- 서비스의 규모가 매우 커질 가능성이 높다.(2)
- SQL 재사용이 드물거나 중요하지 않다.(2)
- SQL을 **app 의 일부**로 관리한다. (5)
- **DB 를 직접 열어 확인해야한다.** (5)

- 나는 EF 를 극혐한다. (10)
- 내 SQL 이 좀 찼다. (10)
- **DBA** 와 친하게 지내고 싶다. (10)

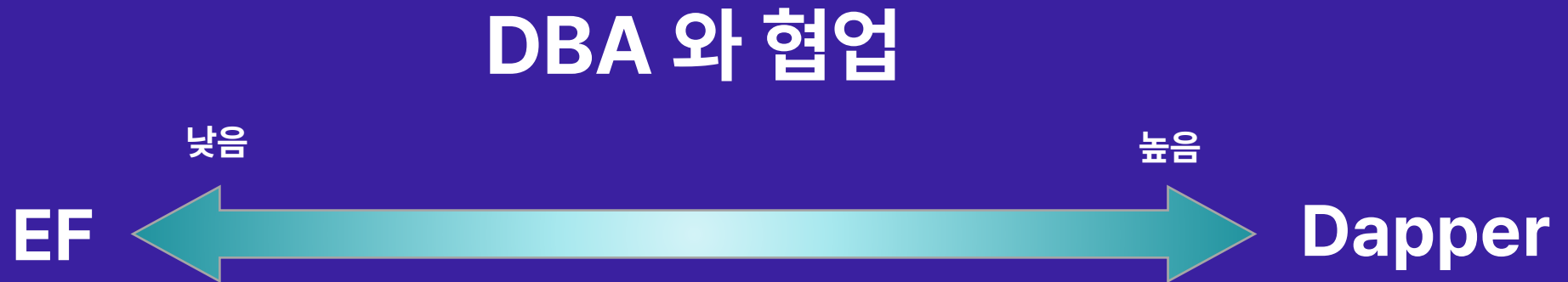
그래서 당신의 선택은?

- 한마디로 기준 정리



그래서 당신의 선택은?

- 한마디로 기준 정리



그래서 당신의 선택은?

- 한마디로 기준 정리



이제 해야 할 일들..

이제 해야 할 일들..

- 뭘 선택하든 이것은 꼭 하자.
 - Repository 패턴 적용하기.
 - 테스트 작성하기.

이제 해야 할 일들..

- Repository 패턴 적용하기

- Dapper 는 Repository 패턴이 필수.
 - DB Context 를 추상화 하거나 주입하는 형태를 지원하지 않음.
 - 매 사용 위치에서 connection string 생성하는 코드를 쓸 수는 없음.
 - 매 사용 위치마다 SQL 을 정의 해야함. (SQL 노출 문제)

이제 해야 할 일들..

```
public interface IUserRepository
{
    Task<User?> GetByIdAsync(int id);
    Task<User?> GetByEmailAsync(string email);
    Task<int> CreateAsync(User user);
    Task<bool> UpdateAsync(User user);
    Task<bool> DeleteAsync(int id);
}
```

```
public class UserRepository(IDbConnection connection) : IUserRepository
{
    public async Task<User?> GetByIdAsync(int id)
    {
        var sql = "SELECT * FROM Users WHERE Id = @Id";
        return await connection.QueryFirstOrDefaultAsync<User>(sql, new { Id = id });
    }

    public async Task<User?> GetByEmailAsync(string email)
    {
        var sql = "SELECT * FROM Users WHERE Email = @Email";
        return await connection.QueryFirstOrDefaultAsync<User>(sql, new { Email = email });
    }

    public async Task<int> CreateAsync(User user)
    {
        var sql = @"INSERT INTO Users (Email, Name, IsActive, CreatedAt)
VALUES (@Email, @Name, @IsActive, @CreatedAt);
SELECT CAST(SCOPE_IDENTITY() as int)";
        return await connection.ExecuteScalarAsync<int>(sql, user);
    }

    public async Task<bool> UpdateAsync(User user)
    {
        var sql = @"UPDATE Users SET Email = @Email, Name = @Name, IsActive = @IsActive,
UpdatedAt = @UpdatedAt WHERE Id = @Id";
        var affected = await connection.ExecuteAsync(sql, user);
        return affected > 0;
    }

    public async Task<bool> DeleteAsync(int id)
    {
        var sql = "DELETE FROM Users WHERE Id = @Id";
        var affected = await connection.ExecuteAsync(sql, new { Id = id });
        return affected > 0;
    }
}
```

이제 해야 할 일들..

- **Repository 패턴 적용하기 (+ adv)**

- connection 객체와 실제 DB 호출을 상위 타입으로 위임(base type)
 - base 타입 상속 후 SQL 을 호출 위주로 구현.

- **SQL 을 문자열로 직접 나열하지 말고 partial class 로**

- string 리터럴을 매 위치에서 직접 사용하는 것 지양.
- 별도의 partial class 활용해 string const 로 정의 후 참조.

이제 해야 할 일들..

```

public abstract class BaseRepository(IDbConnection connection)
{
    protected IDbConnection Connection { get; } = connection;

    // 단일 결과 조회
    protected Task<T?> QueryFirstOrDefaultAsync<T>(
        string sql,
        object? param = null,
        IDbTransaction? transaction = null)
        => Connection.QueryFirstOrDefaultAsync<T>(sql, param, transaction);

    // 다중 결과 조회
    protected Task<IEnumerable<T>> QueryAsync<T>(
        string sql,
        object? param = null,
        IDbTransaction? transaction = null)
        => Connection.QueryAsync<T>(sql, param, transaction);

    // 실행 (INSERT, UPDATE, DELETE)
    protected Task<int> ExecuteAsync(
        string sql,
        object? param = null,
        IDbTransaction? transaction = null)
        => Connection.ExecuteAsync(sql, param, transaction);

    // 스칼라 값 반환
    protected Task<T> ExecuteScalarAsync<T>(
        string sql,
        object? param = null,
        IDbTransaction? transaction = null)
        => Connection.ExecuteScalarAsync<T>(sql, param, transaction);

    ...
}

```

```

public partial class UserRepository(IDbConnection connection)
    : BaseRepository(connection), IUserRepository
{
    public Task<User?> GetByIdAsync(int id)
        => QueryFirstOrDefaultAsync<User>(GetByIdQuery, new { Id = id });

    public Task<IEnumerable<User>> GetByEmailAsync()
        => QueryAsync<User>(GetByEmailQuery);

    public Task<int> CreateAsync(User user)
        => ExecuteScalarAsync<int>(UserCreateQuery, user);

    ...
}

public partial class UserRepository
{
    public const string GetByIdQuery =
        """
        SELECT Id, Email, Name, IsActive, CreatedAt, UpdatedAt
        FROM Users WHERE Id = @Id
        """;

    public const string GetByEmailQuery =
        """
        SELECT Id, Email, Name, IsActive, CreatedAt, UpdatedAt
        FROM Users WHERE Email = @Email
        """;

    ...
}

```

이제 해야 할 일들..

- Repository 패턴 적용하기

- EF 도 가능하면 Repository 패턴을 작성하는 게 좋다.
 - EF 는 repository 패턴 필요 없다.. 주장도 있음.
 - 그러나 repository 패턴을 작성하는 것이 구조적으로 안정적.

이제 해야 할 일들..

```
public class UserRepository(AppDbContext context) : IUserRepository
{
    public Task<User?> GetByIdAsync(int id)
    => context.Users
        .Where(SpecificationsById(id))
        .FirstOrDefaultAsync(cancellationToken);

    public Task<User?> GetByEmailAsync(string email)
    => context.Users
        .Where(SpecificationsByEmail(email))

    public async Task<User> CreateAsync(User user)
    {
        context.Users.Add(user);
        await context.SaveChangesAsync(cancellationToken);
        return user;
    }

    public async Task<User> UpdateAsync(User user)
    {
        context.Users.Update(user);
        await context.SaveChangesAsync(cancellationToken);
        return user;
    }

    public async Task<bool> DeleteAsync(int id)
    {
        var user = await GetByIdAsync(id, cancellationToken);
        if (user is null)
            return false;

        context.Users.Remove(user);
        await context.SaveChangesAsync(cancellationToken);
        return true;
    }
}
```

이제 해야 할 일들..

- 테스트 코드 작성하기

- 어떤 형태로든 테스트 코드를 작성하는 게 좋다.
 - 유닛테스트든 통합테스트든 테스트 코드를 남겨두는 것이 좋다.
 - 이 때 repository 패턴을 작성하는 것이 훨씬 도움이 된다.

EF vs Dapper

Let's be **ORM** 깎는 개발자!

Thank you!

